

TÓPICOS DE DESENVOLVIMENTO DE SOFTWARE

Green Software Development

Lara Ferreira PG57579

<https://github.com/178LaPata/Green-Software-Development.git>

Inicialmente, comecei por implementar uma versão **recursiva** e uma versão **linear** da função Fibonacci. De seguida, comparei vários aspetos de **performance** e para além disso, utilizei **otimizações**, mais concretamente a **flag -O2**.

Os testes foram realizados para os números 20, 30, 40, 50, 60, 70, 80 da sequência de Fibonacci.

FUNÇÃO RECURSIVA

```
long fibonacci(int n)
{
    if (n <= 1)
        return n;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}
```

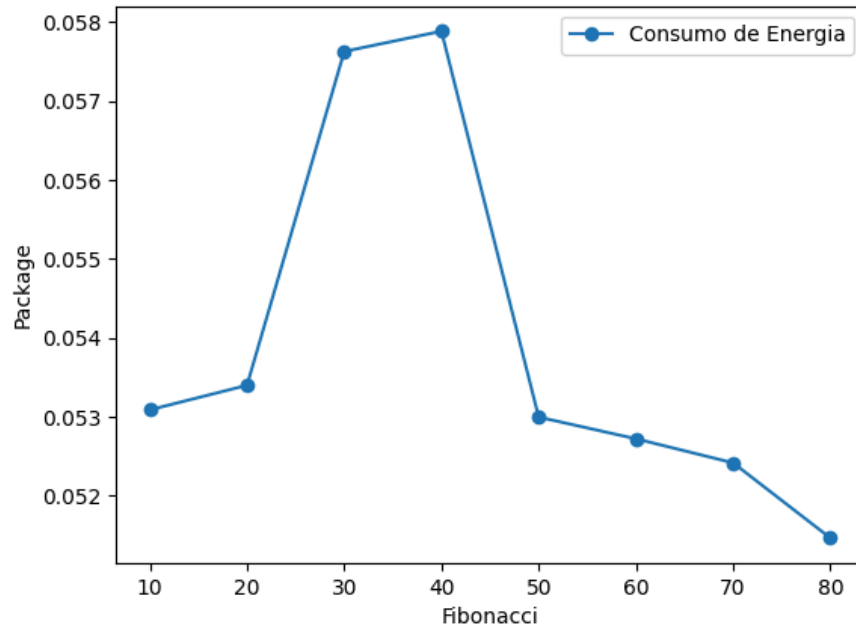


Figure 1: Consumo de energia com Otimização

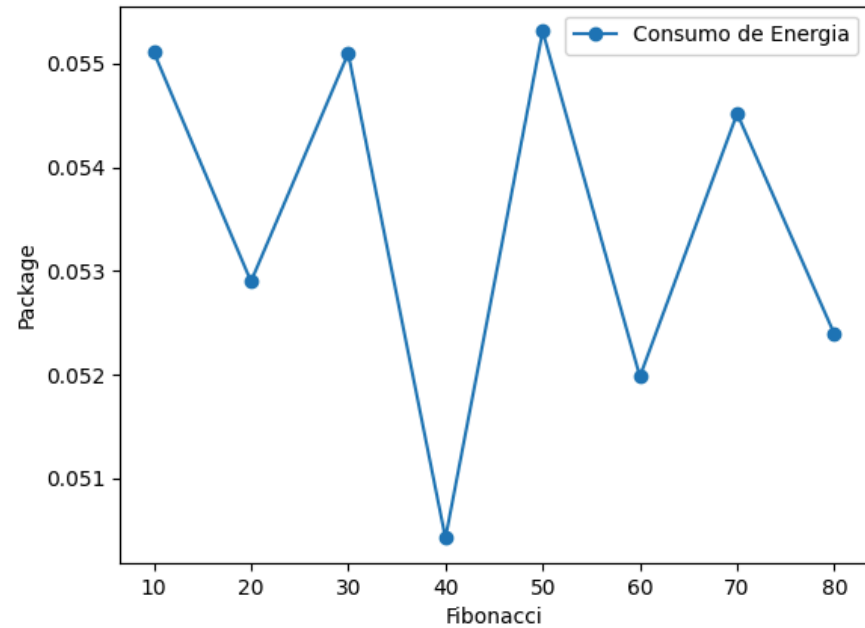


Figure 2: Consumo de energia sem Otimização

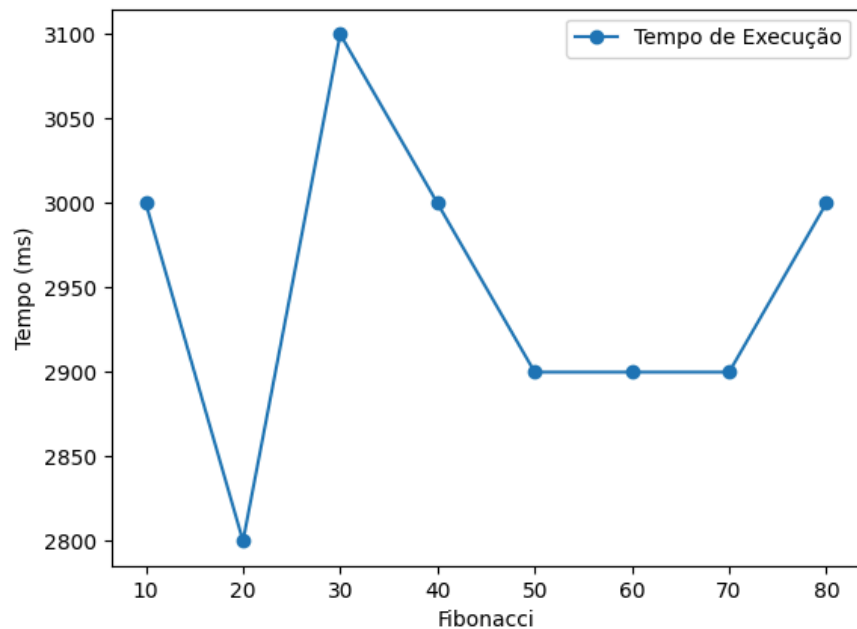


Figure 3: Tempo de execução com Otimização

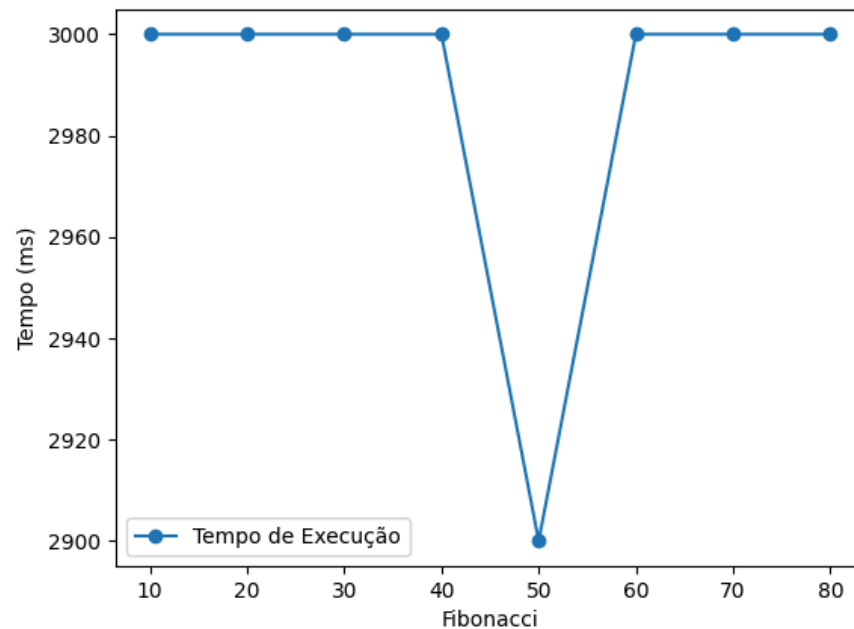


Figure 4: Tempo de execução sem Otimização

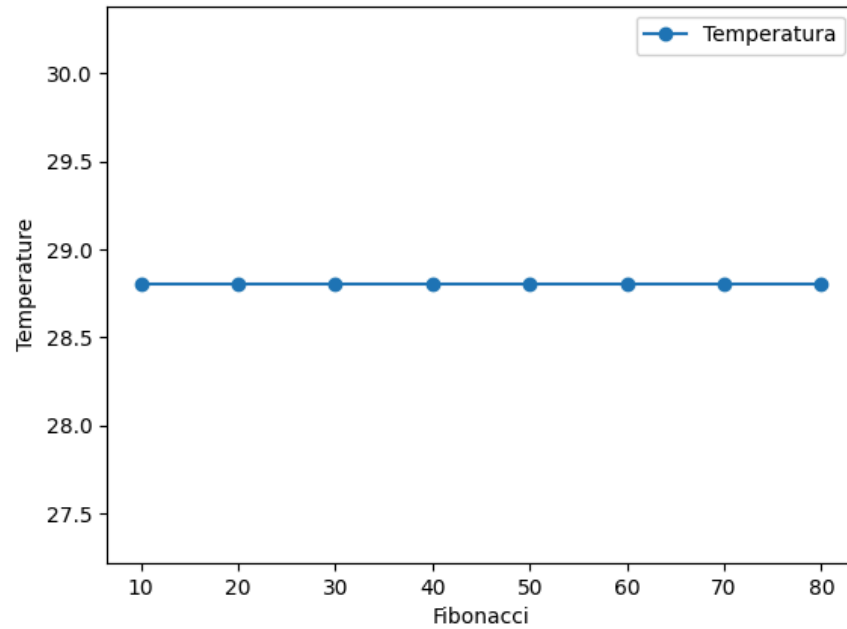


Figure 5: Temperatura com Otimização

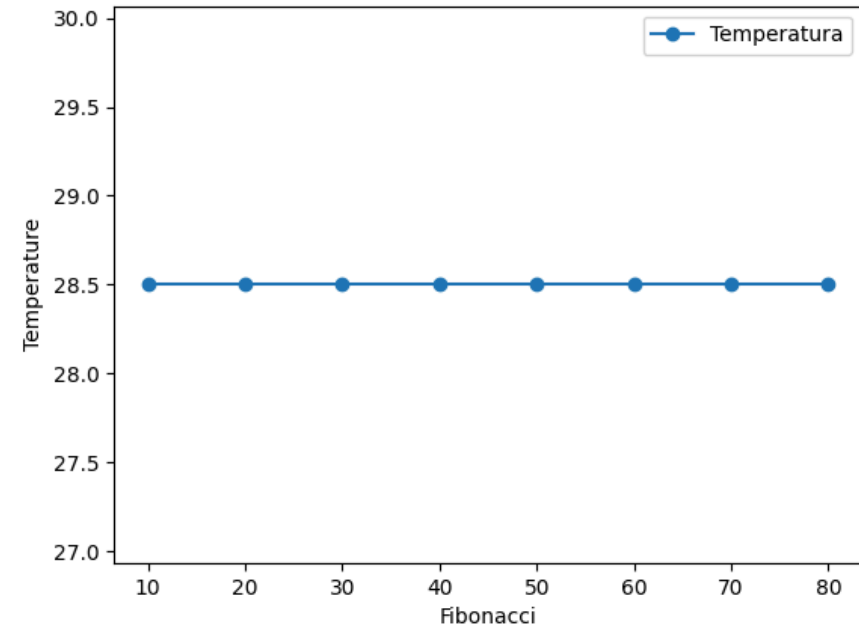


Figure 6: Temperatura sem Otimização

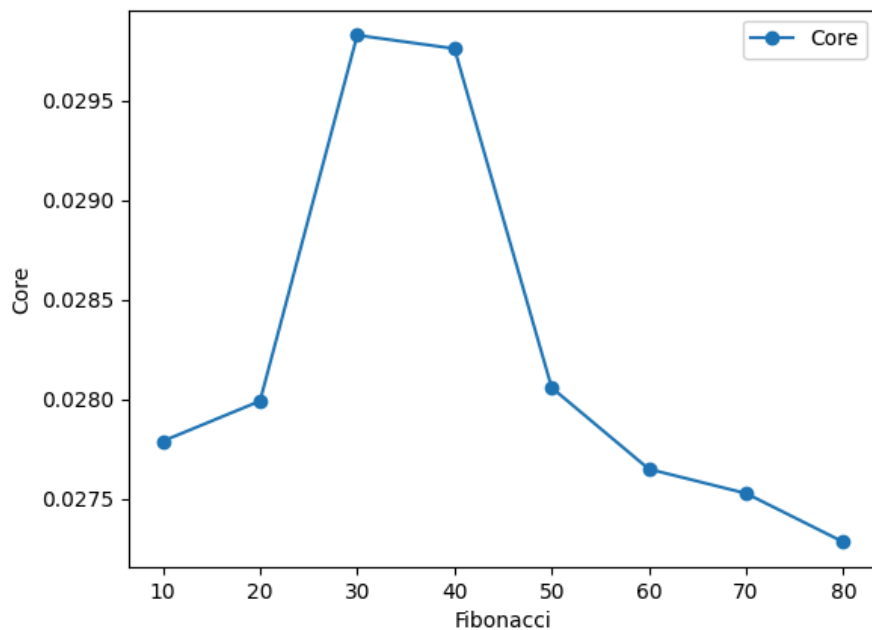


Figure 7: Uso de Core com Otimização

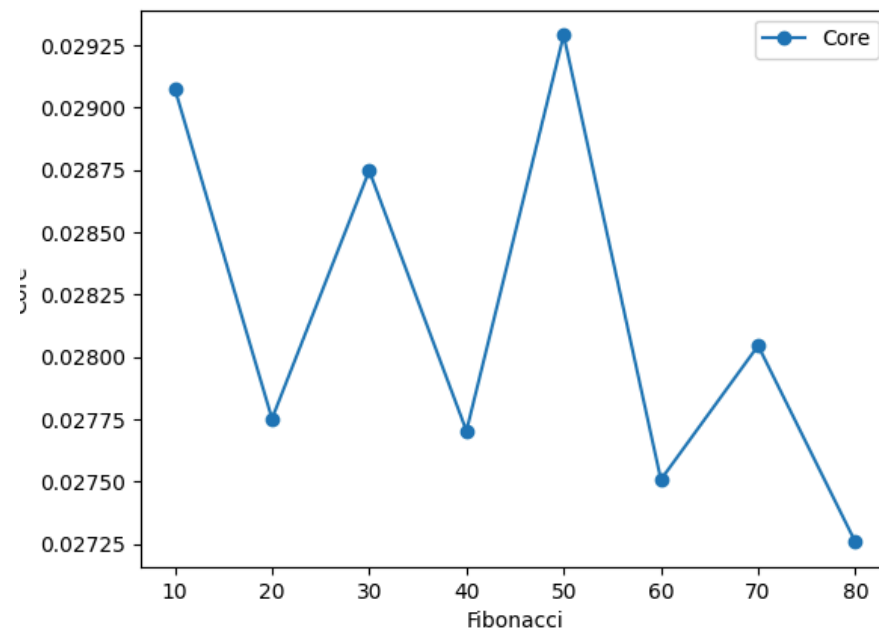


Figure 8: Uso de Core sem Otimização

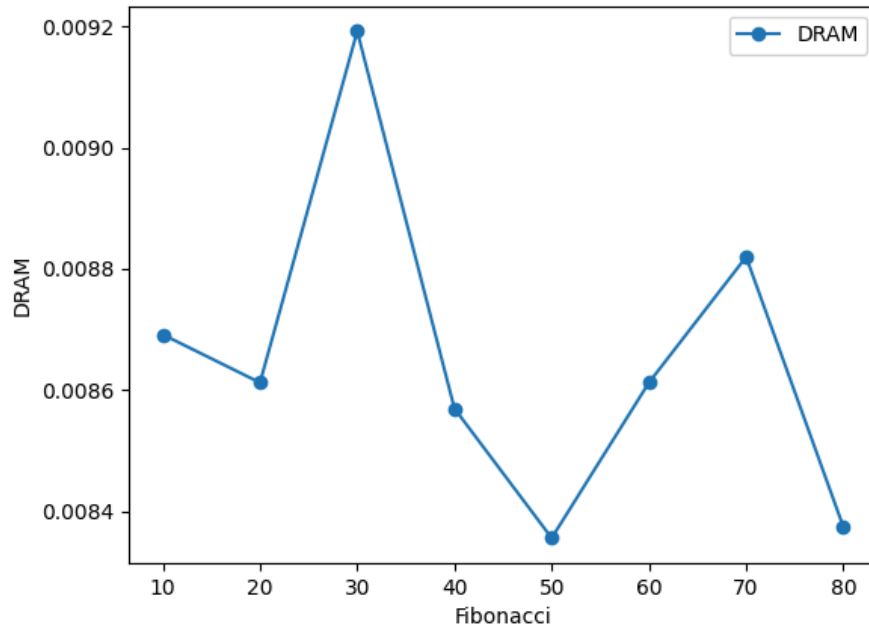


Figure 9: Consumo de DRAM com Otimização

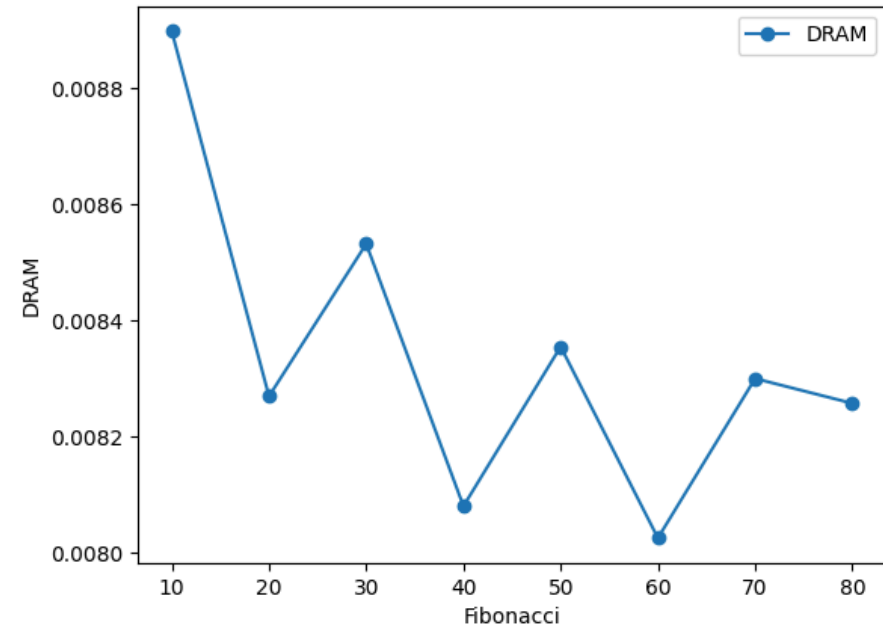


Figure 10: Consumo de DRAM sem Otimização

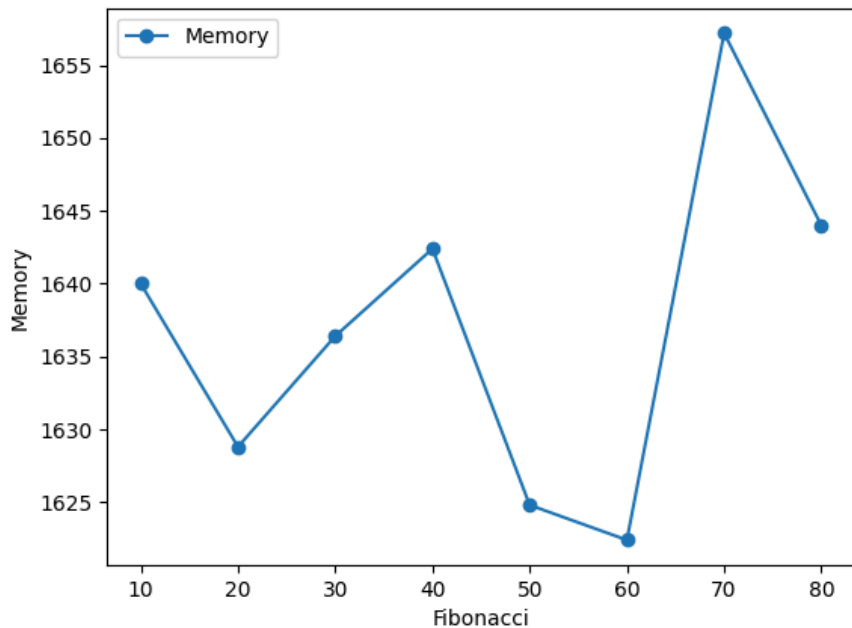


Figure 11: Uso de Memória com Otimização

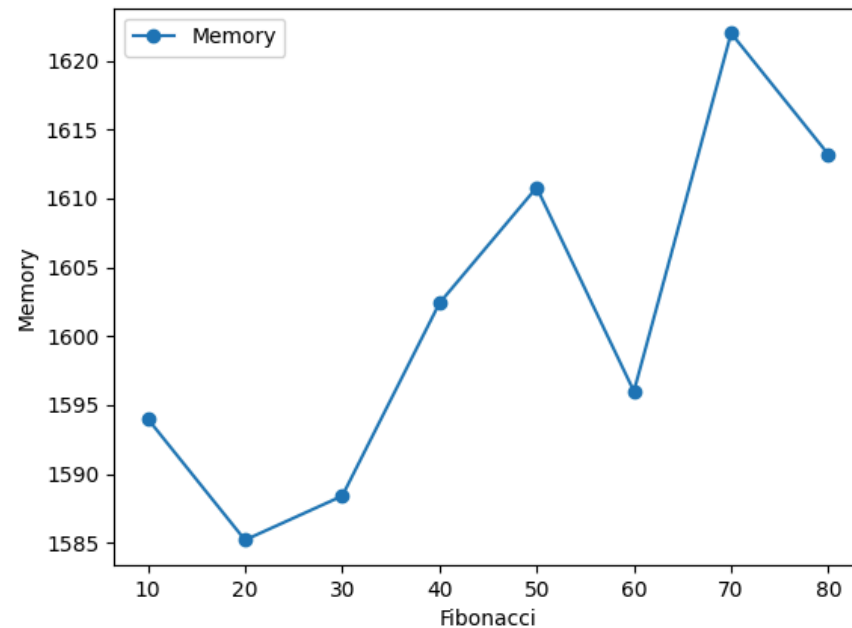


Figure 12: Uso de Memória sem Otimização

A implementação recursiva da função Fibonacci demonstrou as limitações inerentes à sua natureza exponencial $O(2^n)$.

Mesmo tendo sido aplicadas otimizações, esta apresentou um **consumo energético elevado**, uma **alta utilização de recursos** e uma **ineficiência da memória**.

FUNÇÃO LINEAR

```
long fibonacci(int n) {  
    if (n <= 1) return n;  
  
    long a = 0, b = 1, c;  
    for (int i = 2; i <= n; i++) {  
        c = a + b;  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

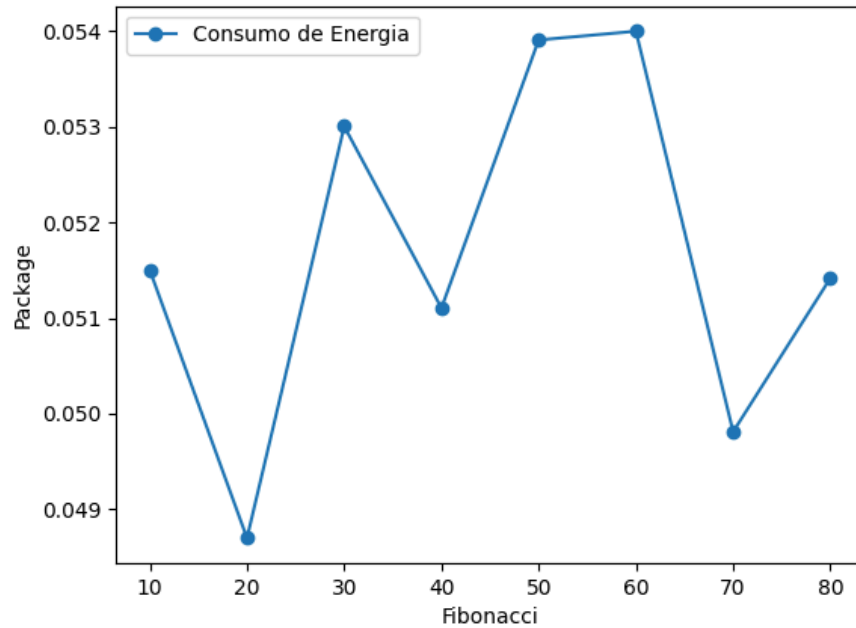


Figure 13: Consumo de energia com Otimização

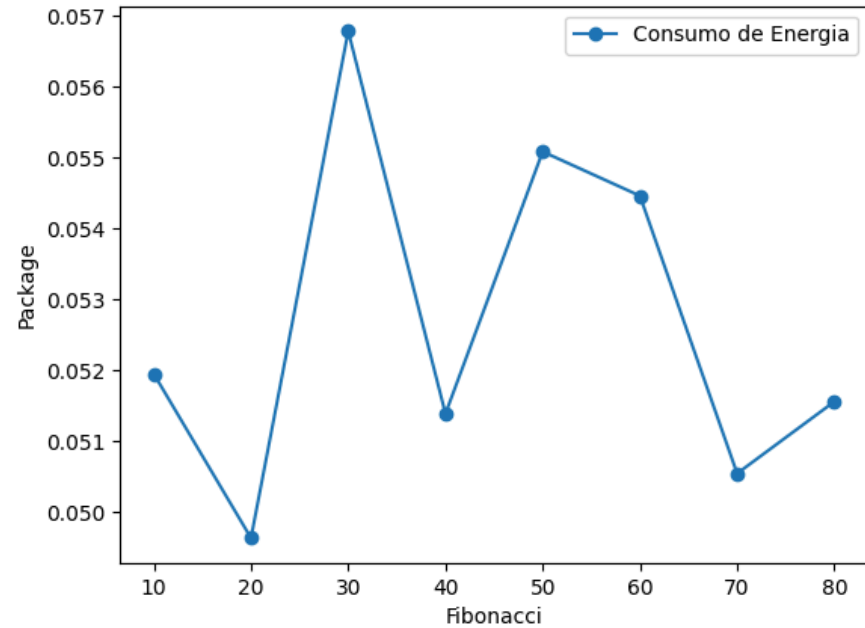


Figure 14: Consumo de energia sem Otimização

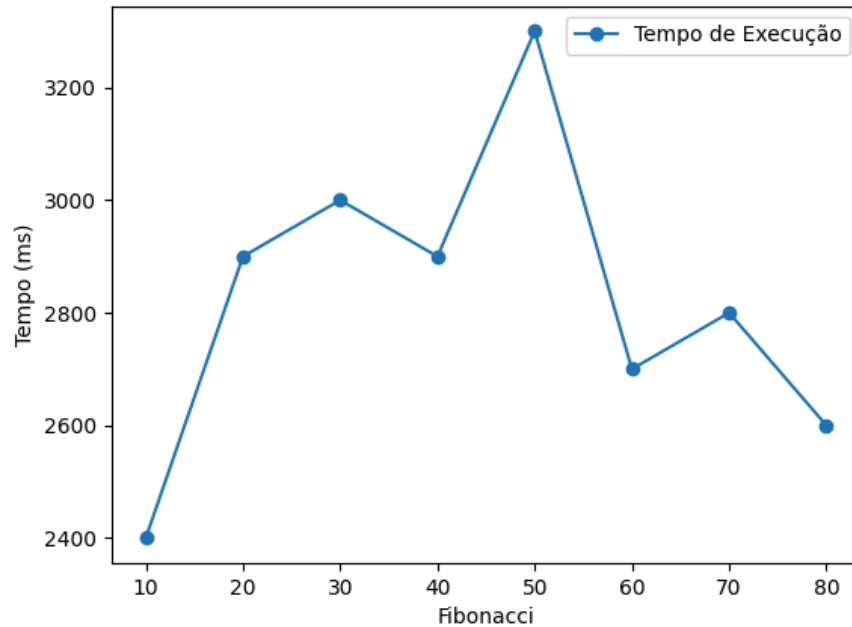


Figure 15: Tempo de execução com Otimização

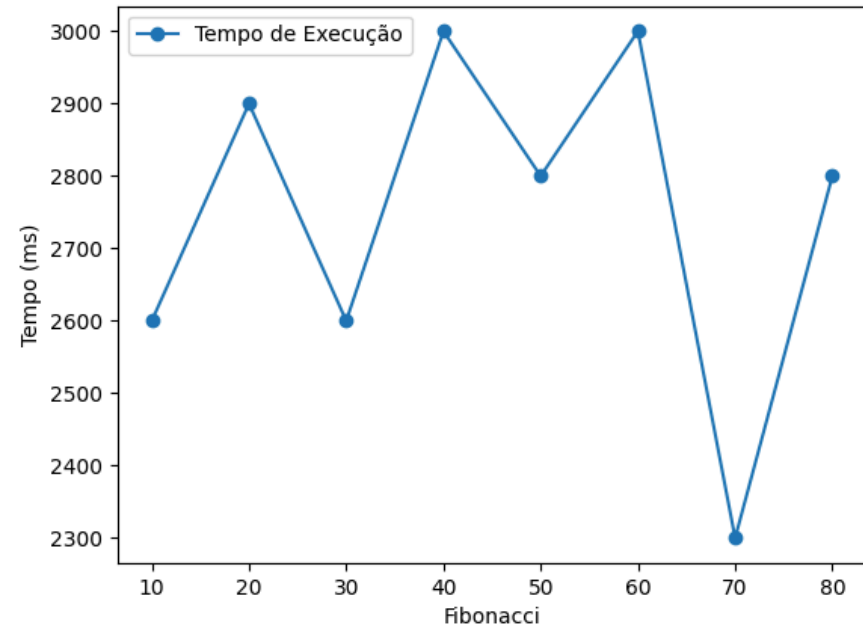


Figure 16: Tempo de execução sem Otimização

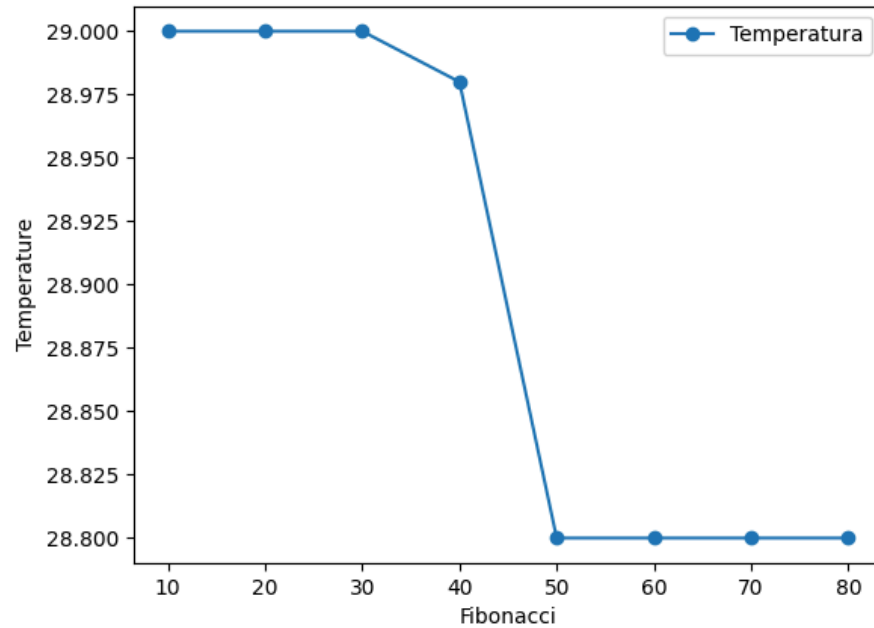


Figure 17: Temperatura com Otimização

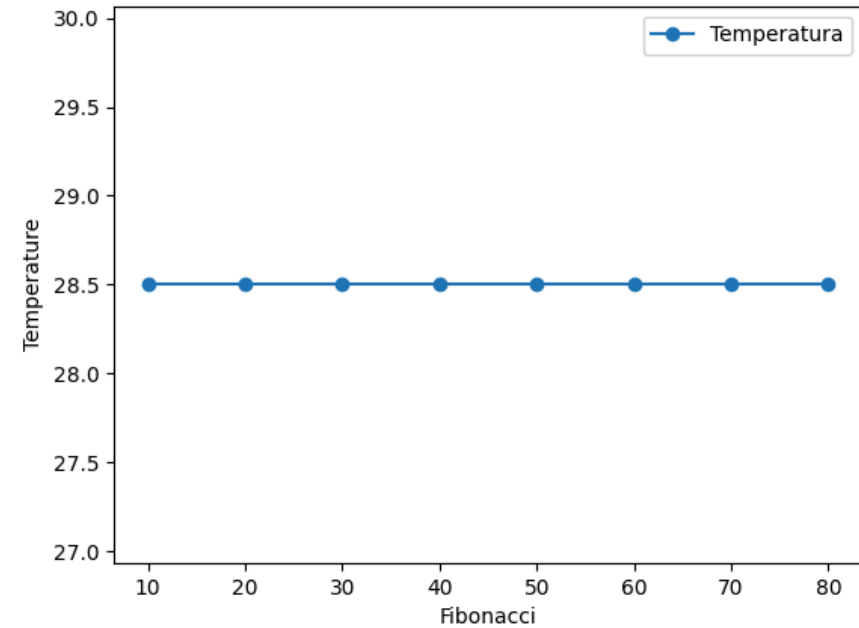


Figure 18: Temperatura sem Otimização

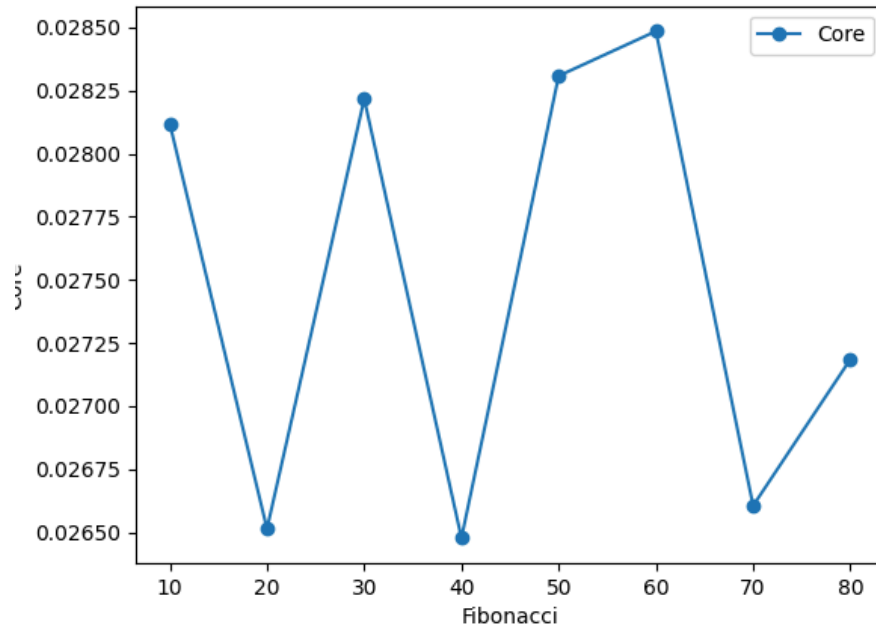


Figure 19: Uso de Core com Otimização

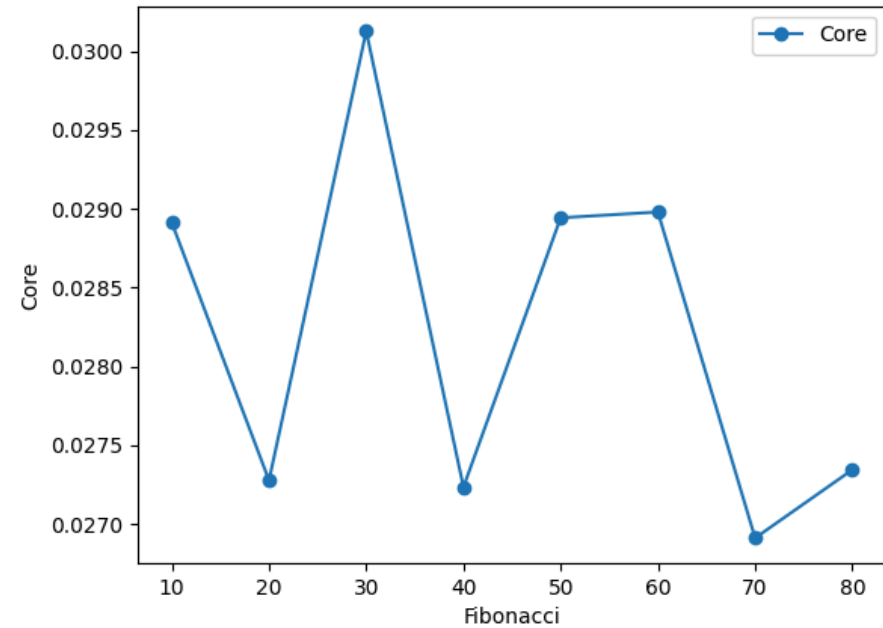


Figure 20: Uso de Core sem Otimização

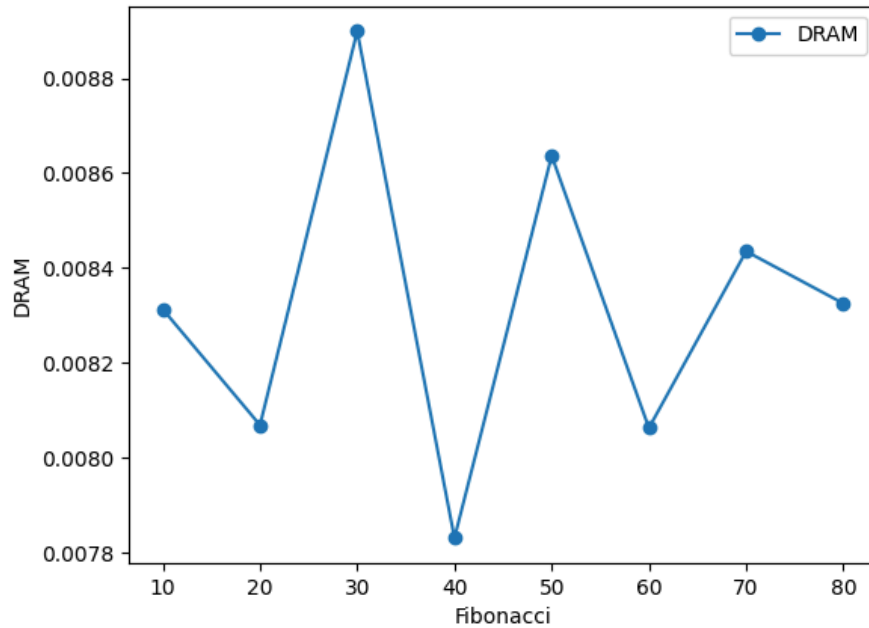


Figure 21: Consumo de DRAM com Otimização

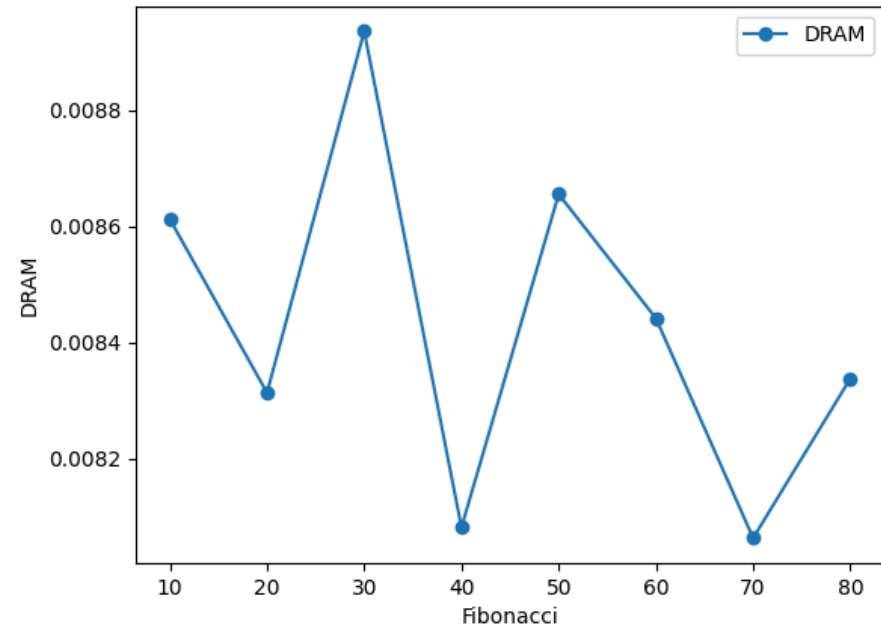


Figure 22: Consumo de DRAM sem Otimização

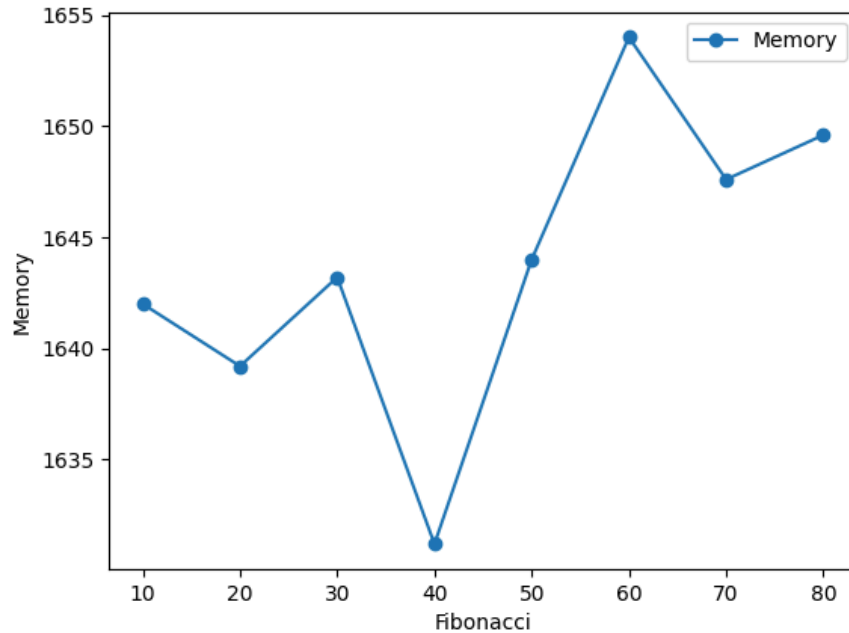


Figure 23: Uso de Memória com Otimização

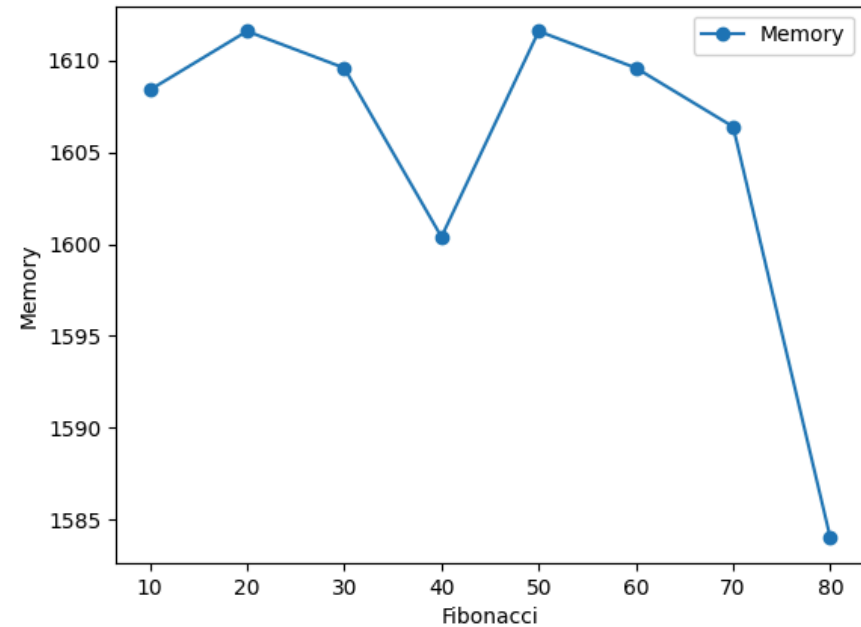


Figure 24: Uso de Memória sem Otimização

A implementação linear da função Fibonacci é extremamente eficiente, tendo complexidade $O(n)$.

Após terem sido aplicadas otimizações, esta apresentou um **consumo energético baixo**, uma **baixa utilização de recursos** e uma **eficiência da memória**.

CONCLUSÃO

As otimizações do compilador demonstraram **benefícios** substanciais em ambas as implementações:

- Na implementação **recursiva**, as otimizações reduziram o consumo energético em aproximadamente **75%** e **melhoraram** significativamente o desempenho, embora **não** alterassem a natureza exponencial do algoritmo.
- Na implementação **linear**, as otimizações proporcionaram melhorias adicionais de **40-50%** no consumo energético e tempo de execução, otimizando um algoritmo já **eficiente**.

Tal como podemos verificar, **não é suficiente** aplicar as otimizações, sendo necessário fazer uma análise para cada caso **individualmente**.

Comparando as implementações recursiva e linear do algoritmo de Fibonacci, é possível observar que a escolha do algoritmo tem um impacto **significativamente maior** que as otimizações, uma vez que a implementação linear $O(n)$ **superou** consideravelmente a recursiva $O(2^n)$ em todos os parâmetros analisados, mesmo quando esta última **utilizava** otimizações.